

LAPORAN POSTTEST 5
PRAKTIKUM BASIS DATA



Disusun oleh:
Anugerah Fakhriza Reswara (2509106025)
Kelas (A2'25)

PROGRAM STUDI INFORMATIKA
UNIVERSITAS MULAWARMAN
SAMARINDA

2026

DAFTAR ISI

DAFTAR ISI.....	2
BAB 1 PENJELASAN POSTTEST 4.....	4
1.1 Penjelasan.....	4
1.2 Tujuan.....	4
BAB 2 PENJELASAN ENTITAS DAN ATRIBUT.....	2
2.1 Entitas dan Atribut.....	2
2.1.1 Entitas Pelanggan (Tabel: pelanggan).....	2
2.1.2 Entitas Transaksi (Tabel: transaksi).....	2
BAB 3 IMPLEMENTASI DAN ANALISIS HASIL.....	4
3.1 Inisialisasi dan persiapan data.....	4
3.1.1 Pembuatan Skema Database.....	4
3.1.2 Pengisian Data Sampel (Seeding).....	4
3.1.3 Output setelah pengisian data sampel.....	6
3.2 Penghitungan Baris menggunakan COUNT.....	7
Query A: Menghitung Populasi Pelanggan Dewasa.....	7
Query B: Audit Transaksi Bernilai Tinggi.....	8
3.3 Penjumlahan Nilai menggunakan SUM.....	8
Query A: Kondisi Umur < 30 Tahun.....	8
Query B: Total Pendapatan Kotor (Gross Revenue).....	9
3.4 Rata-rata nilai menggunakan AVG.....	9
Query A: Rata-rata Usia Pelanggan.....	9
Query B: Average Order Value (AOV).....	9
3.5 Nilai Ekstrim menggunakan MIN/MAX.....	10
Query A: Identifikasi Saldo Terendah.....	10
Query B: Rekor Transaksi Tertinggi.....	10
3.6 Pengelompokan Data menggunakan GROUP BY & HAVING.....	12
Query A: Analisis Akumulasi Penjualan Harian Berbasis Target.....	12

Query B: Analisis Penjualan Harian Berbasis Target.....	12
3.7 Logika Kondisional menggunakan CASE.....	13
BAB 4 KESIMPULAN.....	14
4.1 Ringkasan Teknis.....	14
4.2. Nilai Strategis Penguasaan Query.....	14
BAB 5 PENUTUP.....	15
5.1 Penutup.....	15
5.2 Saran Pengembangan.....	15

BAB 1

PENJELASAN POSTTEST 4

1.1 Penjelasan

Dalam era transformasi digital, data telah menjadi aset yang sangat berharga bagi setiap organisasi. Namun, data mentah yang tersimpan dalam jumlah besar tidak akan memberikan manfaat maksimal tanpa adanya kemampuan untuk memproses dan menganalisisnya secara efisien. Basis data relasional muncul sebagai solusi untuk mengorganisir informasi secara sistematis, memungkinkan akses cepat, pengelolaan yang terstruktur, dan pembaruan data yang konsisten.

SQL (Structured Query Language) adalah bahasa standar yang digunakan untuk berinteraksi dengan Sistem Manajemen Basis Data Relasional (RDBMS). Penguasaan SQL tidak hanya terbatas pada perintah dasar seperti Create, Read, Update, dan Delete (CRUD), tetapi juga mencakup teknik manipulasi data tingkat lanjut. Penggunaan subquery, operator set, dan filter string yang kompleks sangat krusial dalam dunia kerja nyata, misalnya untuk menghasilkan laporan keuangan, analisis perilaku pelanggan, hingga optimasi stok barang yang memerlukan penggabungan informasi dari berbagai sumber tabel secara logis dan efisien.

1.2 Tujuan

Tujuan utama dari posttest 4 ini adalah untuk memberikan pemahaman praktis serta pengalaman langsung dalam:

- Mengimplementasikan perintah NOT IN dikombinasikan dengan teknik subquery untuk menyaring data berdasarkan kriteria yang dinamis dari tabel lain.
- Menerapkan mekanisme pengurutan data (Ordering) secara descending dengan filter logika tingkat lanjut guna menyajikan data yang informatif.
- Memahami dan mempraktikkan penggunaan operator ALL untuk perbandingan data skalar terhadap kumpulan nilai (result set).

- Menganalisis penggunaan operator UNION untuk menggabungkan dua atau lebih hasil query menjadi satu kesatuan informasi yang utuh.
- Melakukan eksplorasi filter teks menggunakan operator NOT LIKE dengan berbagai pola wildcard guna melakukan pencarian negatif atau pembersihan data dari pola yang tidak diinginkan.

BAB 2

PENJELASAN ENTITAS DAN ATRIBUT

2.1 Entitas dan Atribut

Entitas adalah komponen utama pada level Logical Model yang merepresentasikan objek atau konsep di dunia nyata yang informasinya ingin disimpan sedangkan Atribut adalah elemen data yang mendefinisikan karakteristik atau properti spesifik dari sebuah entitas.

2.1.1 Entitas Pelanggan (Tabel: pelanggan)

Tabel ini berfungsi sebagai master data yang menyimpan informasi fundamental mengenai individu yang terdaftar dalam sistem. Berikut adalah atribut-atribut yang ada di tabel pelanggan:

- **id (VARCHAR 5)**: Bertindak sebagai Primary Key. Penggunaan tipe data VARCHAR memberikan fleksibilitas jika identitas menggunakan kombinasi huruf dan angka.
- **nama (VARCHAR 50)**: Menyimpan nama lengkap. Atribut NOT NULL memastikan setiap entitas wajib memiliki identitas nama.
- **alamat (VARCHAR 100)**: Informasi domisili untuk keperluan logistik atau korespondensi.
- **umur (INT)**: Data numerik usia yang digunakan untuk segmentasi pasar atau analisis demografi.
- **saldo (DECIMAL 10,2)**: Menyimpan nilai finansial. Tipe Decimal digunakan untuk menghindari isu presisi pada angka pecahan yang sering terjadi pada tipe Float.
- **tanggal_bergabung (DATE)**: Rekam jejak loyalitas pelanggan sejak pertama kali terdaftar.

2.1.2 Entitas Transaksi (Tabel: transaksi)

Tabel ini mencatat setiap histori pembelian yang terjadi, mencerminkan aktivitas ekonomi dalam sistem. Berikut adalah atribut-atribut yang ada di tabel transaksi:

- **id (VARCHAR 10)**: Identitas unik setiap transaksi (misal: kode struk).
- **total_pembelian (DECIMAL 12,2)**: Nilai belanja bruto dalam satu sesi transaksi.
- **waktu_transaksi (TIMESTAMP)**: Mencatat waktu secara presisi hingga satuan detik, memudahkan

analisis waktu puncak (peak hour) transaksi.

BAB 3

IMPLEMENTASI DAN ANALISIS HASIL

3.1 Inisialisasi dan persiapan data

Proses implementasi dimulai dengan tahap krusial yaitu membangun lingkungan fisik database melalui perintah Data Definition Language (DDL). Langkah ini tidak hanya sekedar membuat wadah penyimpanan, tetapi juga menetapkan aturan main atau business rules yang harus dipatuhi oleh setiap data yang akan masuk ke dalam sistem.

3.1.1 Pembuatan Skema Database

Perintah CREATE DATABASE digunakan untuk mengalokasikan ruang nama (namespace) terisolasi di dalam RDBMS. Hal ini memastikan bahwa tabel-tabel yang dibuat tidak akan berbenturan dengan proyek lain dalam satu server yang sama. Penggunaan perintah USE setelahnya sangat penting untuk menetapkan konteks kerja aktif, sehingga instruksi selanjutnya dieksekusi pada basis data yang tepat.

3.1.2 Pengisian Data Sampel (Seeding)

Tahap terakhir dalam persiapan adalah pengisian data awal menggunakan perintah Data Manipulation Language (DML) yaitu INSERT INTO. Data yang dimasukkan dirancang sedemikian rupa untuk mencakup berbagai skenario pengujian, seperti variasi umur, rentang saldo yang ekstrem (dari ribuan hingga puluhan juta), serta tanggal bergabung yang tersebar untuk menguji validitas query pencarian berdasarkan waktu.

```
-- Membuat database baru dan menggunakannya
CREATE DATABASE POSTTEST_5;
USE POSTTEST_5;

-- Mendefinisikan struktur tabel pelanggan
CREATE TABLE pelanggan (
    id VARCHAR (5) PRIMARY KEY,
    nama VARCHAR (50) NOT NULL,
    alamat VARCHAR (100),
```



```

        umur INT NOT NULL,
        saldo DECIMAL(10,2) NOT NULL,
        tanggal_bergabung DATE NOT NULL
    );

-- Mendefinisikan struktur tabel transaksi
CREATE TABLE transaksi (
    id VARCHAR(10) PRIMARY KEY,
    total_pembelian DECIMAL(12,2) NOT NULL,
    waktu_transaksi TIMESTAMP NOT NULL
);

-- Memasukkan data sampel ke tabel pelanggan
INSERT INTO pelanggan (id, nama, alamat, umur, saldo, tanggal_bergabung)
VALUES
    ('1', 'Anugerah Fakhriza Reswara', 'Jalan In Aja', 18, 10000.00,
    '2025-08-01');

INSERT INTO pelanggan (id, nama, alamat, umur, saldo, tanggal_bergabung)
VALUES
    ('2', 'Muhammad Alfauzi Syahputra', 'Jalan Harapan Baru', 10,
    50000000.00, '2023-02-11');

INSERT INTO pelanggan (id, nama, alamat, umur, saldo, tanggal_bergabung)
VALUES
    ('3', 'Pirlo Syabila Hafuza', 'Jalan Praktek', 24, 110000.00,
    '2024-04-02');

INSERT INTO pelanggan (id, nama, alamat, umur, saldo, tanggal_bergabung)
VALUES
    ('4', 'Yoga Ananda Prasetya', 'Jalan Teori', 11, 220000.00,
    '2025-05-20');

INSERT INTO pelanggan (id, nama, alamat, umur, saldo, tanggal_bergabung)
VALUES
    ('5', 'Akbar Rachim', 'Jalan Yang Diatas', 19, 500000.00,
    '2026-01-31');

INSERT INTO pelanggan (id, nama, alamat, umur, saldo, tanggal_bergabung)
VALUES

```

```
( '6', 'Denny Mulia', 'Jalan Sepuh', 45, 71500.00, '2022-12-15'),
( '7', 'Muhammad Zidane Abdul Kadir', 'Jalan Luas', 50, 923010.00,
'2021-05-20');
```

3.1.3 Output setelah pengisian data sampel

Setelah dilakukannya pengisian sampel pada 2 tabel tersebut, untuk melihat data yang sudah dimasukkan, saya akan menggunakan query `SELECT * FROM namatabel;`

```
SELECT * FROM pelanggan;
```

	id	nama	alamat	umur	saldo	tanggal_bergabung
1	1	Anugerah Fakhriza Reswara	Jalan In Aja	18	10000.00	2025-08-01
2	2	Muhammad Alfauzi Syahputra	Jalan Harapan Baru	10	50000000.00	2023-02-11
3	3	Pirlo Syabila Hafuza	Jalan Praktek	24	110000.00	2024-04-02
4	4	Yoga Ananda Prasetya	Jalan Teori	11	220000.00	2025-05-20
5	5	Akbar Rachim	Jalan Yang Diatas	19	500000.00	2026-01-31
6	6	Denny Mulia	Jalan Sepuh	45	71500.00	2022-12-15
7	7	Muhammad Zidane Abdul Kadir	Jalan Luas	50	923010.00	2021-05-20

Gambar 3.1.4.1 Isi Data Dari Tabel Pelanggan

```
SELECT * FROM transaksi;
```

	id	total_pembelian	waktu_transaksi
1	ORDER001	200000.00	2025-11-22 08:33:44
2	ORDER002	150241.00	2025-11-04 21:22:33
3	ORDER003	56040.00	2026-02-02 15:30:23
4	ORDER004	44420.00	2026-03-04 14:11:00
5	ORDER005	190931.00	2026-03-15 22:55:21

Gambar 3.1.4.2 Isi Data Dari Tabel Transaksi

3.2 Penghitungan Baris menggunakan COUNT

Fungsi COUNT sangat esensial untuk audit jumlah data atau menghitung populasi dalam kategori tertentu. Perintah ini memfilter tabel secara horizontal terlebih dahulu (WHERE umur > 18), lalu menghitung baris yang tersisa. Ini memberikan gambaran mengenai ukuran pasar potensial yang secara hukum dianggap dewasa.

Query A: Menghitung Populasi Pelanggan Dewasa

Perintah ini memfilter tabel secara horizontal terlebih dahulu WHERE umur > 18, lalu menghitung baris yang tersisa. Ini memberikan gambaran mengenai ukuran pasar potensial yang secara hukum dianggap dewasa. Hasil: Berdasarkan data sampel, terdapat 4 orang yang memenuhi kriteria ini.

```
SELECT COUNT(*) AS total_dewasa FROM pelanggan WHERE umur > 18;
```

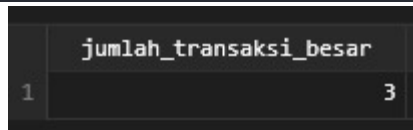
	jumlah_pelanggan_dewasa
1	4

Gambar 3.2 Output Query A

Query B: Audit Transaksi Bernilai Tinggi

Di sini, fungsi COUNT berfungsi sebagai detektor volume. Dalam manajemen inventaris atau keuangan, mengetahui berapa banyak pesanan yang bernilai tinggi (di atas 100.000) membantu dalam alokasi sumber daya pengiriman atau prioritas layanan pelanggan. Hasil eksekusi menunjukkan terdapat 3 transaksi high-value.

```
SELECT COUNT(*) AS jumlah_transaksi_besar FROM transaksi WHERE  
total_pembelian > 100000;
```



	jumlah_transaksi_besar
1	3

Gambar 3.2 Output Query B

3.3 Penjumlahan Nilai menggunakan SUM

Fungsi COUNT() digunakan untuk menghitung frekuensi atau jumlah kemunculan record dalam sebuah tabel berdasarkan kriteria yang diberikan dalam klausa WHERE.

Query A: Kondisi Umur < 30 Tahun

Query ini menjumlahkan aset pelanggan di lokasi geografis tertentu. Jika nilai ini signifikan dibandingkan area lain, area tersebut dapat diidentifikasi sebagai kluster ekonomi tinggi. Dalam database riil, penggunaan SUM sangat sensitif terhadap nilai NULL, namun karena struktur tabel kita menggunakan NOT NULL, hasil kalkulasi dipastikan akurat secara absolut.

```
SELECT SUM(saldo) AS total_saldo_jalan_in_aja FROM pelanggan WHERE alamat =  
'Jalan In Aja';
```



total_saldo_jalan_in_aja
10000.00

Gambar 3.3 Output Query A

Query B: Total Pendapatan Kotor (Gross Revenue)

Ini adalah query fundamental untuk laporan keuangan. Nilai ini menunjukkan total aliran kas masuk (cash inflow) ke perusahaan. Tanpa fungsi SUM, analis harus menjumlahkan manual ribuan baris transaksi, yang berisiko pada human error.

```
SELECT SUM(total_pembelian) AS total_pendapatan FROM transaksi;
```

total_pendapatan
641632.00

Gambar 3.3 Output Query B

3.4 Rata-rata nilai menggunakan AVG

Fungsi AVG() bekerja dengan menjumlahkan seluruh nilai (SUM) lalu membaginya dengan jumlah baris (COUNT). Hasilnya memberikan nilai representatif atau titik tengah dari dataset.

Query A: Rata-rata Usia Pelanggan

Rata-rata usia memberikan wawasan demografis. Jika rata-rata usia cenderung rendah (misal di bawah 20), maka strategi produk harus disesuaikan dengan tren anak muda. Pada data ini, rata-rata umur pelanggan berada pada angka 25.28.

```
SELECT AVG(umur) AS rata_rata_umur FROM pelanggan;
```

rata_rata_umur
25.2857

Gambar 3.4 Output Query A

Query B: Average Order Value (AOV)

Dalam dunia bisnis, AOV (Average Order Value) adalah metrik vital. Query ini menunjukkan berapa rata-rata uang yang dikeluarkan pelanggan dalam satu kali transaksi. Jika AOV meningkat dari waktu ke waktu, itu menandakan strategi upselling perusahaan berhasil.

```
SELECT AVG(total_pembelian) AS rata_rata_pembelian FROM transaksi;
```

rata_rata_pembelian
128326.400000

Gambar 3.4 Output Query B

3.5 Nilai Ekstrim menggunakan MIN/MAX

Fungsi MIN() dan MAX() digunakan untuk mencari nilai batas bawah dan batas atas. Fungsi ini bekerja dengan melakukan perbandingan (comparison) antar nilai dalam satu kolom.

Query A: Identifikasi Saldo Terendah

Digunakan untuk memonitor batas minimum simpanan. Secara teknis, query ini hanya membutuhkan satu kali lintasan data untuk menentukan nilai terkecil.

```
SELECT MIN(saldo) AS saldo_paling_sedikit FROM pelanggan;
```

saldo_paling_sedikit
10000.00

Gambar 3.5 Output Query A

Query B: Rekor Transaksi Tertinggi

Membantu dalam mengidentifikasi "Whale Customer" atau transaksi terbesar yang pernah dicapai. Informasi ini penting untuk menetapkan limit transaksi di masa depan atau memberikan apresiasi

kepada pelanggan dengan belanja tertinggi.

```
SELECT MAX(total_pembelian) AS pembelian_termahal FROM transaksi;
```

pembelian_termahal
200000.00

Gambar 3.5 Output Query B

3.6 Pengelompokan Data menggunakan GROUP BY & HAVING

Klausa GROUP BY mereduksi baris-baris data menjadi kelompok unik, sedangkan HAVING melakukan filter setelah grup tersebut terbentuk (setelah fungsi agregat dijalankan).

Query A: Analisis Akumulasi Penjualan Harian Berbasis Target

Query ini bertujuan mendeteksi nilai umur yang muncul lebih dari satu kali (non-unique values). Jika hasil query ini kosong, hal tersebut mengindikasikan bahwa data pada kolom umur bersifat unik untuk setiap pelanggan (tidak ada dua pelanggan atau lebih yang memiliki usia yang sama dalam dataset saat ini). Secara teknis, klausa HAVING membuang semua grup yang hasil COUNT(*)-nya sama dengan 1.

```
SELECT umur, COUNT(*) AS jumlah_orang FROM pelanggan GROUP BY umur HAVING COUNT(*) > 1;
```

```
MariaDB [POSTTEST_5]> SELECT umur, COUNT(*) AS jumlah_orang FROM pelanggan GROUP BY umur HAVING COUNT(*) > 1;  
Empty set (0.001 sec)
```

Gambar 3.6 Output Query A

Query B: Analisis Penjualan Harian Berbasis Target

Fungsi DATE() menyatukan transaksi pada hari yang sama. Klausa HAVING digunakan untuk memfilter hari-hari produktif yang mencapai target penjualan (di atas 100.000), sangat berguna untuk laporan KPI harian.

```
SELECT DATE(waktu_transaksi) AS tanggal, SUM(total_pembelian) AS total_harian FROM transaksi GROUP BY tanggal HAVING total_harian > 100000;
```

tanggal	total_harian
2025-11-04	150241.00
2025-11-22	200000.00
2026-03-15	190931.00

Gambar 3.6 Output Query B

3.7 Logika Kondisional menggunakan CASE

Transformasi data menjadi kategori yang mudah dipahami melalui evaluasi kondisi secara dinamis (row-by-row). Penggunaan ekspresi CASE memungkinkan kita menciptakan kolom virtual yang memberikan interpretasi kontekstual terhadap data mentah tanpa perlu mengubah struktur fisik tabel secara permanen.

Secara teknis, ekspresi CASE beroperasi layaknya logika if-then-else dengan melakukan evaluasi linear per baris hingga kondisi pertama terpenuhi. Implementasi ini sangat vital dalam pelaporan Business Intelligence untuk mentransformasi angka abstrak menjadi kategori fungsional yang siap dipresentasikan kepada stakeholder. Dengan label deskriptif ini, data menjadi lebih informatif bagi pengguna non-teknis dan memudahkan tim pemasaran dalam menyusun strategi promosi yang spesifik untuk setiap segmen pelanggan tanpa memerlukan pengolahan tambahan di level aplikasi.

```
SELECT nama, umur,  
       CASE  
         WHEN umur < 13 THEN 'Anak-anak'  
         WHEN umur BETWEEN 13 AND 19 THEN 'Remaja'  
         WHEN umur BETWEEN 20 AND 45 THEN 'Dewasa'  
         ELSE 'Lansia'  
       END AS kategori_usia  
FROM pelanggan;
```

nama	umur	kategori_usia
Anugerah Fakhri Reswara	18	Remaja
Muhammad Alfauzi Syahputra	10	Anak-anak
Pirlo Syabila Hafuza	24	Dewasa
Yoga Ananda Prasetya	11	Anak-anak
Akbar Rachim	19	Remaja
Denny Mulia	45	Dewasa
Muhammad Zidane Abdul Kadir	50	Lansia

Gambar 3.7 Output Query

BAB 4

KESIMPULAN

4.1 Ringkasan Teknis

Berdasarkan hasil praktikum, fungsi agregasi SQL bekerja secara efisien dalam meringkas dataset besar. Penggunaan GROUP BY memungkinkan organisasi data berdasarkan parameter tertentu, sementara HAVING memberikan kontrol filtrasi pada level grup yang tidak dapat dicapai oleh WHERE. Secara teknis, mesin database memproses query ini dengan membentuk partisi data sementara sebelum menjalankan fungsi agregat, yang memastikan akurasi pada metrik seperti total pendapatan harian atau frekuensi demografis.

4.2. Nilai Strategis Penguasaan Query

Penguasaan terhadap fungsi-fungsi ini memberikan nilai strategis bagi pengembang dan analis. Kemampuan untuk mengonversi data mentah menjadi kategori fungsional melalui CASE dan mendeteksi tren melalui agregasi memungkinkan pembuatan laporan Business Intelligence yang dinamis. Hal ini meminimalkan ketergantungan pada pemrosesan manual di sisi aplikasi, sehingga mempercepat waktu pengambilan keputusan berdasarkan data aktual yang tersimpan dalam sistem.

BAB 5

PENUTUP

5.1 Penutup

Laporan posttest ini mendokumentasikan keberhasilan penggunaan fungsi agregat dan logika kondisional dalam SQL untuk menjawab tantangan analisis data. Pemahaman mengenai perbedaan WHERE dan HAVING serta optimalisasi GROUP BY menjadi pondasi penting dalam membangun sistem basis data yang informatif. Seluruh query yang dijalankan telah berhasil menyajikan informasi yang relevan bagi kebutuhan operasional maupun manajerial.

5.2 Saran Pengembangan

Untuk meningkatkan performa dan kedalaman analisis di masa mendatang, disarankan agar dilakukan penerapan Indexing pada kolom-kolom yang sering dijadikan parameter pengelompokan (seperti waktu_transaksi). Selain itu, integrasi antara fungsi agregat dengan perintah JOIN dapat dieksplorasi lebih lanjut untuk menarik wawasan integratif dari berbagai tabel yang saling berelasi dalam skema database yang lebih kompleks.